

Laboratorio di Calcolo

A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Variabili e tipi

Che cos'è una variabile?

Una **variabile** è una zona di memoria a cui diamo un **nome**, che contiene un **valore** di un certo **tipo**.

- **Nome:** l'identificatore con cui la usiamo nel codice (`massa`).
- **Tipo:** dice quanti byte occupa e **come interpretare** i bit (intero? virgola mobile?).
- **Valore:** il contenuto attuale, che può **cambiare** durante l'esecuzione.
- **Indirizzo:** la posizione in memoria (lo useremo con i puntatori).

Memoria (schematica)

0×7ffc10	`int n = 42`
0×7ffc14	*libero: 4 byte disponibili*
0×7ffc18	`double massa = 1.67e-27`
0×7ffc1c	(un double occupa 8 byte)
0×7ffc20	`float t = 0.25`

Gli stessi 32 bit valgono `1078530011` se letti come `int` e `3.1415927` se letti come `float` : è il **tipo** a dare significato ai bit.

Dichiarazione

In C ogni variabile deve essere **dichiarata prima di essere usata**, specificandone il tipo.

```
tipo nome;
```

```
int n_particelle;
double energia;
double x, y, z;       // tre variabili
```

- La dichiarazione **riserva la memoria** e associa il nome al tipo.
- Il tipo di una variabile **non può cambiare** dopo la dichiarazione.

Usare una variabile non dichiarata

```
int main() {
    tempo = 2.5;
    return 0;
}
```

```
error: ‘tempo’ undeclared
```

Inizializzazione e assegnamento

Inizializzazione

Valore assegnato **nella dichiarazione**:

```c
double g = 9.81;
int n = 0, m = 10;
```

Assegnamento

Nuovo valore a una variabile **già esistente**:

```c
double t;
t = 0.0;
t = t + 0.1;   // !!
```

`=` **non** è l'uguaglianza matematica!

Significa: *calcola l'espressione a destra e scrivi il risultato nella variabile a sinistra*.

Per questo `t = t + 1` ha senso, e `5 = t` è un errore.

Variabili non inizializzate

```c
#include <stdio.h>

int main() {
    int somma;            // valore?
    somma = somma + 5;
    printf("%d\n", somma);
    return 0;
}
```

```
$ ./somma
32772          # oppure 5, oppure ...
```

Il valore iniziale è **indeterminato**: dipende da cosa c'era prima in quella memoria. Inizializzate **sempre** le variabili (e `-Wall` vi avviserà).

Scegliere i nomi delle variabili

Valgono le regole sugli identificatori viste nella lezione 8. In più, alcune **convenzioni**:

- **Nomi che descrivono il significato:** `velocita` invece di `x2`.
- **Stile coerente:** in C si usa spesso `snake_case` (`tempo_volo`, `n_passi`).
- **Notazione della fisica** quando è chiara: `v0`, `dt`, `t_max`, `omega`.
- **Unità di misura** nel nome o nel commento: `massa_kg`, oppure `double m; // [kg]`.
- Nomi brevi (`i`, `j`, `n`) solo per contatori e indici.

Risultato dell'esame: bocciato!

```
1   double a, b, c, d;
2   a = 9.81;
3   d = b * c - 0.5 * a * c * c;
```

Meglio così:

```
1   double g = 9.81;      // [m/s^2]
2   double v0 = 15.0;     // [m/s]
3   double t = 1.2;       // [s]
4   double h = v0 * t - 0.5 * g * t * t;
```

I tipi fondamentali del C

Tipo	Byte*	Intervallo / precisione	Uso tipico
`char`	1	$-128 \dots 127$	Caratteri
`short`	2	$-32\,768 \dots 32\,767$	Interi piccoli
`int`	4	$\approx \pm 2.1 \times 10^{9}$	**Interi (default)**
`long`	8	$\approx \pm 9.2 \times 10^{18}$	Interi grandi
`long long`	8	$\approx \pm 9.2 \times 10^{18}$	Interi grandi
`float`	4	$\approx 10^{\pm 38}$, ~7 cifre significative	Reali, poca memoria
`double`	8	$\approx 10^{\pm 308}$, ~16 cifre significative	**Reali (default)**

* Valori tipici su Linux a 64 bit, come in laboratorio. Lo standard fissa solo dei minimi: ad esempio su Windows un `long` occupa 4 byte.

- Gli interi usano il **complemento a due** (lezione 3); i reali lo standard **IEEE 754** (lezione 4).
- Nel calcolo scientifico usate `int` per contare e `double` per misurare.

L'operatore sizeof

`sizeof` restituisce il numero di **byte** occupati da un tipo o da una variabile.

```c
#include <stdio.h>

int main() {
    double x = 1.0;

    printf("char:   %zu\n", sizeof(char));
    printf("int:    %zu\n", sizeof(int));
    printf("long:   %zu\n", sizeof(long));
    printf("float:  %zu\n", sizeof(float));
    printf("double: %zu\n", sizeof(double));
    printf("x:      %zu\n", sizeof(x));

    return 0;
}
```

```
$ ./dimensioni
char:   1
int:    4
long:   8
float:  4
double: 8
x:      8
```

- `sizeof` si applica sia ai **tipi** sia alle **variabili**.
- `%zu` è il segnaposto per il risultato di `sizeof`.
- Fate girare questo programma in laboratorio: cosa ottenete?

Interi con e senza segno

- Con n bit, gli interi **con segno** vanno da -2^{n-1} a $2^{n-1}-1$.
- Con `unsigned`, gli interi **senza segno** vanno da 0 a 2^n-1.
- I limiti sono definiti in `<limits.h>`: `INT_MAX`, `INT_MIN`, `UINT_MAX`, …
- Un risultato fuori intervallo produce **overflow** (lezione 3):
  - `unsigned`: si "riavvolge" (aritmetica modulo 2^n).
  - con segno: comportamento **indefinito**, da evitare sempre.

```c
#include <stdio.h>
#include <limits.h>

int main() {
    unsigned int u = 0;
    int n = INT_MAX;

    u = u - 1;
    n = n + 1;

    printf("u = %u\n", u);
    printf("n = %d\n", n);
    return 0;
}
```

```
$ ./overflow
u = 4294967295
n = -2147483648
```

Tipi in virgola mobile

- `float` (32 bit) e `double` (64 bit) seguono lo standard **IEEE 754**.
- La precisione è **limitata**: molti numeri decimali "semplici" non sono rappresentabili esattamente.
- `float` ha circa 7 cifre significative: oltre $2^{24} = 16\,777\,216$ non distingue più tutti gli interi.
- **Usate** `double` salvo motivi specifici: costa poco e riduce molto gli errori di arrotondamento.

```c
#include <stdio.h>

int main() {
    float f = 16777217;
    float d = 0.1f;

    printf("%f\n", f);
    printf("%.10f\n", d);
    printf("%.17f\n", 0.1 + 0.2);
    return 0;
}
```

```
$ ./precisione
16777216.000000
0.1000000015
0.30000000000000004
```

`%.10f` stampa 10 cifre decimali: lo vedremo meglio con `printf`.

Costanti letterali

Anche i valori scritti direttamente nel codice hanno un **tipo**.

Letterale	Tipo	Note
`42` , `-7`	`int`	Decimale
`42u`	`unsigned int`	Suffisso `u`
`42L`	`long`	Suffisso `L`
`0x2A`	`int`	Esadecimale = 42
`3.14`	`double`	**Default per i reali**
`3.14f`	`float`	Suffisso `f`
`6.022e23`	`double`	6.022×10^{23}
`'A'`	`char` (int)	Codice ASCII 65

Attenzione alle trappole

- `2` è un `int` , `2.0` e `2.` sono `double` : la differenza conta (tra poco vediamo perché).
- Uno **zero iniziale** indica la base **ottale**:

```
1   int a = 010;   // a vale 8, non 10!
```

- `'A'` (apici singoli) è un **carattere**; `"A"` (virgolette doppie) è una **stringa**.
  - Una stringa è una **sequenza di caratteri** terminata dal carattere nullo `'\0'` : `"A"` occupa quindi **2 byte** (`'A'` e `'\0'`). Le studieremo più avanti.

Caratteri: char è un intero

- Un `char` contiene un **numero intero** di 1 byte.
- Il numero corrisponde a un carattere secondo la tabella **ASCII** (`'A'` = 65, `'a'` = 97, `'0'` = 48).
- Si può quindi fare **aritmetica** sui caratteri.
- Con `printf`, `%c` stampa il carattere e `%d` il suo codice numerico.
- Ritroveremo i caratteri parlando di stringhe.

```c
#include <stdio.h>

int main() {
    char c = 'A';

    printf("%c %d\n", c, c);
    printf("%c\n", c + 1);
    return 0;
}
```

```
$ ./caratteri
A 65
B
```

Costanti: const

Alcune grandezze **non devono cambiare** durante il programma: costanti fisiche, parametri fissi.

```
1   const double G = 6.674e-11;   // [N m^2 / kg^2]
2   const double c = 2.998e8;     // [m/s]
3   const int N_PASSI = 1000;
4
5   c = 3.0e8;   // errore!
```

```
1   error: assignment of read-only variable ‘c’
```

- `const` chiede al compilatore di **impedire** ogni modifica successiva.
- Una variabile `const` va **inizializzata** nella dichiarazione.
- Trasforma un errore logico (costante modificata per sbaglio) in un errore di **compilazione**, molto più facile da trovare.
- Vedremo un’alternativa, `#define`, quando parleremo del preprocessore.

Conversioni implicite

Quando un'espressione mescola tipi diversi, il C **converte automaticamente**.

Nelle espressioni: verso il tipo "più ampio"

`int` → `long` → `float` → `double`

```c
int    n = 3;
double x = 0.5;
double y = n * x;   // n diventa 3.0: y = 1.5
```

Nell'assegnamento: verso il tipo a **sinistra**

```c
int a = 3.99;    // a vale 3
int b = -3.99;   // b vale -3
```

- Da reale a intero la parte decimale viene **troncata**, non arrotondata.
- Questa conversione **perde informazione** e `-Wall -Wextra` **non** vi avvisa!
- Per ricevere un avviso aggiungete l'opzione `-Wconversion`:

```
$ gcc -Wall -Wextra -Wconversion -o conv conv.c
conv.c:4:13: warning: conversion from ‘double’
  to ‘int’ changes value from ‘3.9900000000000002e+0’
  to ‘3’ [-Wfloat-conversion]
```

La trappola della divisione intera

- Se **entrambi** gli operandi sono interi, `/` esegue la **divisione intera**: il risultato è troncato.
  - `7 / 2` vale `3`
  - `-7 / 2` vale `-3`
  - `7.0 / 2` vale `3.5`
- Il tipo della variabile a sinistra **non conta**: la divisione avviene *prima* dell'assegnamento.

```
1   double x = 1 / 3;   // x vale 0.0 !
```

Un classico errore in fisica

```
1   double m = 2.0;   // [kg]
2   double v = 3.0;   // [m/s]
3
4   double K1 = 1/2 * m * v * v;
5   double K2 = 0.5 * m * v * v;
```

```
1   K1 = 0.000000
2   K2 = 9.000000
```

`1/2` è una divisione tra interi: vale **0**. Scrivete `0.5`, `1.0/2` oppure `m * v * v / 2`.

Conversioni esplicite: il *cast*

Possiamo forzare una conversione scrivendo il tipo tra parentesi: `(tipo) espressione`.

```c
int a = 5, b = 2;

double r1 = a / b;
double r2 = (double) a / b;
double r3 = (double) (a / b);
```

```
r1 = 2.000000
r2 = 2.500000
r3 = 2.000000
```

- `r1`: divisione intera, poi conversione: **2.0**.
- `r2`: il cast si applica **solo ad** `a` (ha precedenza sulla divisione), che diventa `5.0`; poi `5.0 / 2` = **2.5**
- `r3`: le parentesi fanno eseguire **prima** la divisione intera; il cast arriva troppo tardi: **2.0**.
- Uso tipico: calcolare la **media** di valori interi.

```c
double media = (double) somma / n_misure;
```

Mettiamo tutto insieme: caduta di un grave

```c
/*
 * caduta.c: altezza di un corpo in caduta libera
 */
#include <stdio.h>

int main() {
    const double g = 9.81;   // [m/s^2]
    double h0 = 20.0;        // [m]
    double t = 1.5;          // [s]

    double h = h0 - 0.5 * g * t * t;
    printf("h(%.2f s) = %.3f m\n", t, h);

    return 0;
}
```

```
$ gcc -Wall -Wextra -o caduta caduta.c
$ ./caduta
h(1.50 s) = 8.964 m
```

- `const` per la costante fisica.
- `double` per tutte le grandezze misurabili.
- Unità di misura nei commenti.
- `0.5` e non `1/2` !
- **Test:** con `t = 0` dobbiamo ottenere `h0` .

Mettetevi alla prova

Cosa stampano queste istruzioni? Pensateci **prima** di andare avanti.

```
1   int    i = 7 / 2;
2   double d = 7 / 2;
3   double e = 7 / 2.0;
4   int    k = 2.9 + 2.9;
5   double m = (double) 1 / 4;
6   int    o = 012;
```

Variabile	Valore	Perché
`i`	`3`	Divisione intera
`d`	`3.0`	Divisione intera, poi conversione
`e`	`3.5`	Un operando è `double`
`k`	`5`	`5.8` troncato
`m`	`0.25`	Cast prima della divisione
`o`	`10`	`012` è ottale: $1 \cdot 8 + 2$